

Intro to Python

 <https://github.com/learn-co-curriculum/phase-3-intro-to-python>  <https://github.com/learn-co-curriculum/phase-3-intro-to-python/issues/new>

Learning Goals

- Understand how to learn a new programming language.
 - Learn the benefits and philosophy of Python as a language.
-

Key Vocab

- **Interpreter:** a program that executes other programs. Python programs require the Python interpreter to be installed on your computer so that they can be run.
 - **Python Shell:** an interactive interpreter that can be accessed from the command line.
 - **Data Type:** a specific kind of data. The Python interpreter uses these types to determine which actions can be performed on different data items.
 - **Exception:** a type of error that can be predicted and handled without causing a program to crash.
 - **Code Block:** a collection of code that is interpreted together. Python groups code blocks by indentation level.
 - **Function:** a named code block that performs a sequence of actions when it is called.
 - **Scope:** the area in your program where a specific variable can be called.
-

Introduction

Throughout your career as a developer, it's highly likely that you'll be expected to learn multiple languages depending on the companies you work for and the kind of projects you're working on. After gaining confidence with one language, like JavaScript, it can feel overwhelming and even scary to start over from scratch with a totally new language.

Thankfully, we can tell you from experience that learning a new programming language isn't so bad! While you were learning JavaScript, you were also learning some other (even more) important skills: how a computer program works; how to **debug code**; how to **search for help** when you're stuck; how to **think like a developer**; and **how you like to learn**.

With those tools at your disposal, learning your second language (and your third, and fourth) will be easier than the first.

To help you on this journey of your second language, Python, this curriculum will start by introducing new syntax and showing the similarities and differences between Python and JavaScript (they have more in common than you may think). Once you've learned the fundamentals, we'll explore what makes Python unique and start building new applications.

As you're learning, it's ok (and completely normal) to make mistakes: every developer that switches between languages forgets syntax (is it `.toUpperCase()` or `.upper()` ?) and needs to [look at the documentation](https://docs.python.org/3/library/stdtypes.html#str.upper)  (<https://docs.python.org/3/library/stdtypes.html#str.upper>) regularly. The most important things are to **write code** to develop your muscle memory, and trust that you'll get better with practice.

What is Python and Where Did it Come From?

Python is an interpreted, object-oriented programming language. Its high-level built-in data structures and dynamic typing make it very useful for fast development of new applications, as well as scripting or "glue" code to combine existing components written in different languages. Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost and complication of long-term program maintenance. Python supports modules and packages for containing code, which encourages program modularity and code reuse. The Python interpreter and the extensive standard library are available in source or binary form without charge on all major platforms and may be freely distributed.

Guido van Rossum began development of Python in 1989 as a side project while working at Centrum Wiskunde & Informatica (CWI) in the Netherlands. Van Rossum thought of Python as a successor to the ABC programming language, one he had helped create earlier in his career. He liked many of the features in ABC, but took issue with many others. Now that he had the skill and opportunity to create his own language, he took his favorite pieces from ABC and added or changed functionality where he saw fit. He gave the new language the name "Python" (after Monty Python's Flying Circus) and named himself "Benevolent Dictator for Life." Despite the authoritarian title, van Rossum has always been a champion of workplace inclusivity, especially with respect to gender equality.

The interpreter and standard library were made available to the public in 1991.

Why Do Developers Love It?

Every programming language was originally designed to solve some kind of problem. For example, JavaScript was created by Brendan Eich to help developers make web pages interactive.

Python was developed because Guido and his team at CWI found it exhausting to develop in C and Unix Shell scripts. Development in these languages was slow, and it took time for even the most experienced engineers to understand code that they hadn't seen before.

I had this idea that given how much time we had available for Amoeba, I could actually build a whole new language, design and implement it from scratch, and then use it to implement our suite of tools and still be ahead of the game compared to a situation where we would have just clunked on writing the things we wanted to write in C.

For three months I did my day job, and at night and whenever I got a chance I kept working on Python. After three months I was to the point where I could tell people, "Look here, this is what I built." It had an interactive interpreter loop, so the first demos were all, "Let's assign an expression to a variable and print it back," or "Let's define a small function and call it," or "Let's put some things in an array and iterate over the array."

My two office mates were almost instantly taken with it and started helping out. A few others within the institute were also excited about Python. We didn't use it on Amoeba right away because it wasn't mature enough to actually develop the system utilities that we wanted. But it worked well on our Unix system, and people outside my department at CWI started using it because it was fun and productive to use.



Though it started as a side project, Python has since grown to be the most used backend language in the world, trailing only the omnipresent JavaScript among all languages. More developers want to learn Python than any other language, and more employers are seeking developers with experience in Python than *any* other language (even JavaScript!)

What Can Python Do?

In the first two phases of the program, you used JavaScript primarily for one thing: to build **client-side** web applications that run in the browser. While it's true that you can use JavaScript to create other kinds of programs as well thanks to Node, we've stayed true to the original intent of the language by using it for frontend development.

Building web applications in JavaScript means we have all kinds of great tools at our disposal thanks to working in the browser environment. We can:

- Make network requests.
- Update the DOM.
- Listen for events.
- Debug our code in the browser's developer tools.

But it also means working in a sandboxed environment. JavaScript's separation from our computers' innermost components means that it can't take full advantage of everything our computers are capable of, like accessing the file system, or connecting directly to a database, or listening for HTTP requests.

Python, on the other hand, **can't** run in the browser. Learning Python means you can build different kinds of programs: **server-side** applications. Writing these kind of applications will mean familiarizing yourself with a new environment. It also means you'll have a new set of tools and features at your disposal, which is awesome! We can use Python to do all sorts of things, like

- Read and write files.
- Listen for network requests and send responses.
- Connect to a database to access and update data.

Thanks to Python's flexibility, it means we can make all kinds of different applications, not just web applications:

- Command line interfaces (CLIs).
- Web servers.
- Games.
- Web scrapers.

Since you'll be learning a new environment, it's important to familiarize yourself not only with Python's syntax, but also how to run and debug Python code. Make sure to code along with the lessons to come so that you can gain confidence in how to not only write code, but also to check what your code is doing and explore all the great features Python has to offer.

Be sure to check out the resources below as well, and bookmark them for future reference!

Resources

- [About Python](https://www.python.org/doc/essays/blurb/) ➞ (https://www.python.org/doc/essays/blurb/).
- [Python documentation](https://docs.python.org/3/) ➞ (https://docs.python.org/3/).
- [Python Style Guide](https://peps.python.org/pep-0008/) ➞ (https://peps.python.org/pep-0008/).
- [Python VSCode Extension](https://code.visualstudio.com/docs/languages/python) ➞ (https://code.visualstudio.com/docs/languages/python).
- [Kite VSCode Extension](https://www.kite.com/) ➞ (https://www.kite.com/).
- [Guido van Rossum: The Early Years of Python](https://www.computer.org/csdl/magazine/co/2015/02/mco2015020007/13rUy3gmYB) ➞ (https://www.computer.org/csdl/magazine/co/2015/02/mco2015020007/13rUy3gmYB).